Group 1 (6 members): FIX Gateway + Order Service (ingestion, validation, persistence)

Member G1-M1: FIX Session & Connectivity (QuickFIX)

- Configure FIX 4.4 session settings (initiator/acceptor), heartbeats, reconnect
- Implement logon/logout, sequence reset, session-level rejects
- Create local test harness with MiniFix/B2BITS simulator configs

Member G1-M2: FIX Message Parsing → Domain Model

- Map FIX NewOrderSingle/Cancel messages to internal Order DTO/domain model
- Validate mandatory tags, type conversions, symbol/side/qty/price rules
- Build error handling: business reject vs session reject rules

Member G1-M3: Order Validation + Enrichment

- Implement validation pipeline (symbol exists, qty range, price ticks, TIF)
- Reference-data lookup hooks (security master/customer master)
- Generate Order Reference Number strategy (unique + sortable)

Member G1-M4: Order Persistence + DB Integration

- Design Order table schema + indexes for high volume (500K–1M orders)
- Implement repository/DAO layer and batch insert strategy
- Implement order status updates (NEW, PARTIAL, FILLED, CANCELLED, REJECTED)

Member G1-M5: Order Event Publishing (WebSocket + Broker)

- Publish OrderReceived/OrderCancelled/OrderUpdated events (to broker)
- Also expose an internal WebSocket stream for low-latency subscriptions
- Define event contracts (JSON schema/versioning) and backpressure policy

Member G1-M6: Order Cache + Performance

- Build in-memory order cache (by symbol, by client, by status)
- Implement cache invalidation/update on DB write & execution updates
- Run load tests for ingestion throughput and latency profiling

Group 2 (6 members): Execution/Trade Service (order book, matching engine, trade booking)

Member G2-M1: In-memory Order Book Manager

- Data structures for bid/ask book per symbol (price-time priority)
- APIs: add, cancel, modify, query top-of-book
- Concurrency strategy (locks vs actor model vs queues) per symbol

Member G2-M2: Matching Engine / Execution Handler

- Implement matching rules (Buy ≥ Sell, partial fills, leaves qty)
- Generate execution events and update order states
- Handle cancel rules (cannot cancel fully executed orders)

Software-Lab-Trading application

Member G2-M3: Trade Generator + Trade Reference

- Create Trade reference number generation (unique, traceable)
- Build trade object creation from matched orders (qty/price/timestamps)
- Implement idempotency to avoid duplicate trade booking on retries

Member G2-M4: Trade Persistence + DB Tuning

- Design Trade table schema + indexes (2M trades) and partitioning approach
- Implement batch writes / async persistence
- Build reconciliation query: trade totals by symbol/time window

Member G2-M5: Trade Event Streaming (WebSocket-first)

- Emit TradeExecuted events to:
    - UI WebSocket aggregator
    - Options pricing service (stream)
    - Telemetry/monitoring consumers
- Implement replay strategy (last N trades per symbol) on client reconnect

Member G2-M6: Integration + Load/Soak Testing

- End-to-end test: Order events → matching → trades
- Stress tests to reach target scale (500K orders / 2M trades)
- Measure and document latency hotspots + optimization recommendations

Group 3 (6 members): Market Data Service + Options Pricing (Black–Scholes) + Reference Data

Member G3-M1: Market Data File Poller

- Implement flat-file reader with scheduled polling interval . Add incremental update process that would mean delta update processing instead of a full refresh after the polling interval
- Parse quotes (symbol, bid/offer/last/close) and validate data
- Produce MarketDataUpdated events

Member G3-M2: Market Data Store + Subscription API

- In-memory latest-price store keyed by symbol (fast O(1) get)
- WebSocket subscription service: subscribe/unsubscribe by symbol
- Snapshot + incremental update protocol for clients

Member G3-M3: Options Pricing Engine (Black–Scholes)

- Implement Black–Scholes pricing with numerical stability checks
- Inputs: underlying price (from trades/market data), static params, volatility/rate
- Unit tests with known reference values and edge cases

Member G3-M4: Trade-driven Pricing Updates

- Consume TradeExecuted stream and recompute option fair price dynamically
- Throttle/aggregate updates (e.g., per symbol per 100ms) to reduce UI churn
- Publish OptionPriceUpdated on WebSocket stream

Member G3-M5: Reference Data + DB Init Service

- Implement DB-init loader for Security Master / Customer Master / static ref files
- Provide cached lookup APIs used by Order validation and UI filters
- Data quality checks + migration scripts

Member G3-M6: Observability for Pricing + Market Data

- Telemetry: message lag, update frequency, dropped events, compute time
- Error logging + dead-letter handling for malformed quote files
- Performance benchmarks (price update rate vs CPU)

Group 4 (6 members): UI + WebSocket Aggregator + System Integration/DevOps/QA

Member G4-M1: WebSocket Gateway/Aggregator

- Single WS endpoint for UI (orders, trades, market data, option prices)
- Route/merge streams from services; implement auth/session management
- Implement topic-based subscriptions (symbol, status, order type)

Member G4-M2: Order Screen UI

- UI components: live order table + filters (symbol/type/status)
- Real-time order status updates via WebSocket events
- Client-side state management (store, pagination, virtual scroll)

Member G4-M3: Cancel Order Workflow

- UI action "Cancel order" → WS request → response handling
- Display failure if order already executed (enforce via server response)
- Audit UI notifications + retry logic on disconnect
- Introduce states such as:
- · Cancel Pending
- · Cancel Rejected
- UI must:
- · Render intermediate states
- · Handle late responses
- · Allow retry
- · This task should also cover audit trail of any order.

Member G4-M4: Options Price Screen UI

- Display fair price results from pricing stream
- Real-time chart/table updates (throttled rendering)
- UX: show "last update time", symbol selection, parameter display

Member G4-M5: Database + Schema + Index Review (cross-team)

- Consolidate schema (orders, trades, security master, customer master, market data)
- Define common keys, FK strategy (or intentionally avoid for speed), indexes
- Provide SQL for common UI queries (by symbol/status/time)

Member G4-M6: CI/CD + End-to-End QA

- Docker compose / local deployment scripts (services + DB + simulator)
- End-to-end test plan: FIX→orders→trades→pricing→UI
- Non-functional tests: throughput, soak, failure recovery, logging checks